

F Prime Platforms

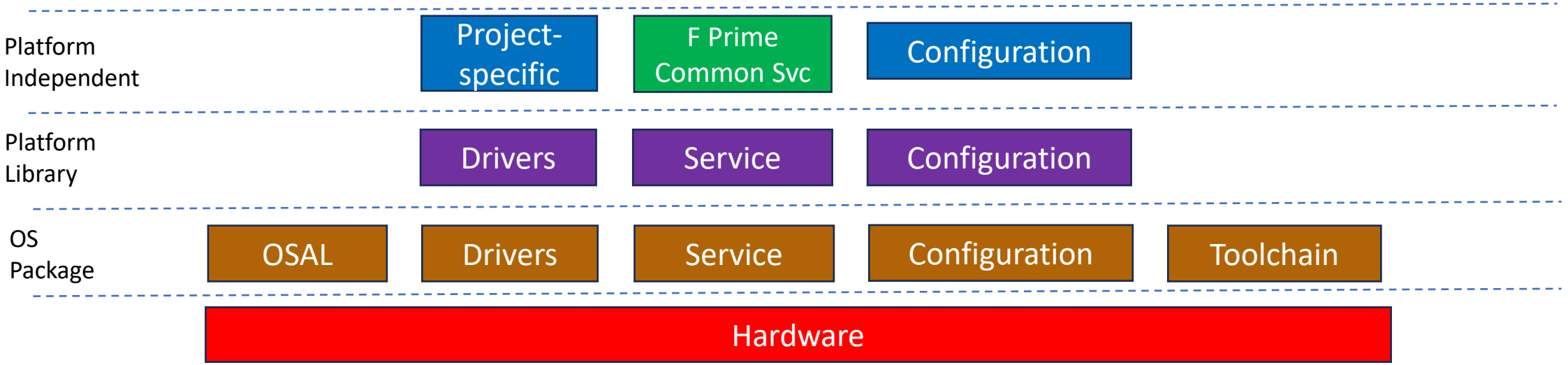
Kevin F. Ortega
NASA Jet Propulsion Laboratory

Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

What's a Supported Platform?

A platform is an Operating System, hardware, and toolchain combination



Supported platform:

- Reference project exists.
- Platform library exists and is used as part of a reference application.
- Tested continuously as part of a continuous integration (CI) pipeline.
- Actively maintained by the F Prime community or core F Prime team.
- Documented build and run instructions are available for users.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Historical and Active Usage

Processor Architecture	Project
x86_64	OWLS
arm32, arm64	Ingenuity/SRH/CADRE
RISC-V	ENGAGE
Sparc	LF/NEAS
PowerPC	VISAR

Operating System	Project
Embedded Linux	Ingenuity/SRH/CADRE
VxWorks 6.7 (F´ v2.x)	LF/NEAS
VxWorks 7	ENGAGE
Zephyr	F´ Infrastructure
FreeRTOS	Community

Standard Platform	Project
Rad 750	VISAR
VORAGO	SRL REAPR
PolarFire SOC	ENGAGE
HPSC	*
Sphinx, Coyote	LF/NEAS
Sabertooth	Div34 Research
Qualcomm SD801	Ingenuity
Qualcomm SD5165 (VOXL2)	SRH
Qualcomm Board (Div33)	QCP
Qualcomm (VOXL1)	CADRE
MicroBlaze (Linux)	CIE
MicroBlaze (Baremetal)	CIE
Baremetal Systems	Ingenuity/SRH

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Currently Supported Platforms

Hardware	OS	Architecture	Reference
Apple Silicon	Darwin	ARM	F Prime Ref
BeagleBoneBlack	VxWorks 7	ARMv7	fprime-vxworks-reference
Feather M4	FreeRTOS	ARM	fprime-featherm4-freertos-reference
RP 2040	Zephyr	ARM	fprime-zephyr-reference
RP 2350	Zephyr	ARM	fprime-zephyr-reference
PolarFire Soc	VxWorks 7	RISC-V	fprime-pfsoc-vxworks7-reference
Raspberry Pi	Linux	ARMv8	fprime-workshop-led-blinker
Teensy41	Zephyr	ARMv7-M	fprime-zephyr-reference
x86	Linux	X86_64	F Prime Ref

https://fprime.jpl.nasa.gov/platforms

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Targeted Platforms (Planned Support)

Hardware	OS	Architecture	Status	Reference
GR740	VxWorks 7	SPARC V8	In development	
HPSC	Linux	RISC-V	In development	fprime-hpsc-linux-reference
HPSC	VxWorks 7	RISC-V	In development	fprime-hpsc-vxworks7-reference
PolarFire SoC	Linux	RISC-V	Not started	
Snapdragon CoProcessor	Linux	ARM	Not started	
STM32	Baremetal	ARM	In development	
Vorago	Baremetal	ARM	In development	fprime-vorago
VOXL2	Linux	ARM64	Not started	

https://fprime.jpl.nasa.gov/platforms

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Targeted Reference

Reference	Status
Hub Reference Project	In development
ROS Reference Project	In development
cFS Reference Project	In development

https://fprime.jpl.nasa.gov/platforms

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development

```
my-library/
├── cmake
│   ├── platform
│   └── toolchain
├── MyLibrary
│   ├── Components
│   │   └── MyComponent
│   │       └── ...
│   ├── Subtopology
│   │   └── ...
│   ├── Types
│   │   └── ...
│   └── Ports
│       └── …
├── library.cmake
└── ci
```

Three requirements:

- `library.cmake` is the only required file in an F Prime library

```
####
# library.cmake for MyLibrary
####
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Components/MyComponent")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Subtopology")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Types")
add_fprime_subdirectory("${CURRENT_CMAKE_LIST_DIR}/MyLibrary/Ports")
```

- Module directories shall be namespaced (i.e. `MyLibrary`) to ensure that modules do not collide with F´ and other libraries. Placing container directories (e.g. `Components`, `Svc`, `Drv`, etc.) directly at the root of the repository is *strongly* forbidden.

- cmake directory to specify platform and toolchain(s)

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

CMakeLists.txt -- provides configuration for platform

```
register_fprime_config(
        MyPlatformTypes
    AUTOCODER_INPUTS
        "${CMAKE_CURRENT_LIST_DIR}/PlatformTypes.fpp"
    HEADERS
        "${CMAKE_CURRENT_LIST_DIR}/PlatformTypes.h"
    INTERFACE # No buildable files generated
)
```

`register_frpime_config` allows projects to override files.

More information https://fprime.jpl.nasa.gov/latest/docs/reference/api/cmake/API/#function-register_fprime_config

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

PlatformTypes.fpp -- defines platform types to be used in this library

- unsigned type of larger sizes internal to the software
  ```
  type PlatformSizeType = U64
  ```
- signed type of larger sizes internal to the software
  ```
  type PlatformSignedSizeType = I64
  ```
- type of smaller indices internal to the software
  ```
  type PlatformIndexType = I16
  ```
- type of arguments to assert functions
  ```
  type PlatformAssertArgType = I32
  ```
- type of task priorities used
  ```
  type PlatformTaskPriorityType = U8
  ```
- type of task identifiers
  ```
  type PlatformTaskIdType = I32
  ```
- type of queue priorities used
  ```
  type PlatformQueuePriorityType = U8
  ```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

PlatformTypes.h-- define platform pointer cast types

```
// Pointer sizes are determined by compiler
#if __SIZEOF_POINTER__ == 8
typedef uint64_t PlatformPointerCastType;
#define PRI_PlatformPointerCastType PRIx64
#elif __SIZEOF_POINTER__ == 4
typedef uint32_t PlatformPointerCastType;
#define PRI_PlatformPointerCastType PRIx32
#elif __SIZEOF_POINTER__ == 2
typedef uint16_t PlatformPointerCastType;
#define PRI_PlatformPointerCastType PRIx16
#elif __SIZEOF_POINTER__ == 1
typedef uint8_t PlatformPointerCastType;
#define PRI_PlatformPointerCastType PRIx8
#else
#error "Expected __SIZEOF_POINTER__ to be one of 8, 4, 2, or 1"
#endif
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

VxWorks.cmake -- mostly to define what OS implementation to use

```
add_fprime_subdirectory(
   "${CMAKE_CURRENT_LIST_DIR}/types/Platform")

register_fprime_config(
        PlatformVxWorks
   INTERFACE # No buildable files generated
   CHOOSES_IMPLEMENTATIONS
        Os_Console_VxWorks
        Os_Mutex_VxWorks
        Os_Queue_VxWorks
        Os_Task_VxWorks
        Fw_StringFormat_snprintf
        # No VxWorks Implementation
        Os_Cpu_Stub
        Os_File_Stub
        Os_Memory_Stub
        Os_RawTime_Stub
   BASE_CONFIG
)
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

In project's setting.ini file, specify which platform to use

```
default_cmake_options: FPRIME_PLATFORM=VxWorks-Posix
```

Reminder: when updating settings.ini, one must regenerate their project.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

<toolchain-file>.cmake -- the toolchain file to specify what tools to use when performing the build as part of CMake

```
set(CMAKE_C_COMPILER clang-20)
set(CMAKE_CXX_COMPILER clang-20)
set(CMAKE_ASM_COMPILER clang-20)

set(COMMON_C_CXX_FLAGS "\
    -fno-exceptions \
    -pedantic \
    -Wextra \
    -Wno-unused-parameter \
    -Werror \
")

set(CMAKE_C_FLAGS "\
    ${COMMON_C_CXX_FLAGS} \
")

set(CMAKE_CXX_FLAGS "\
    ${COMMON_C_CXX_FLAGS} \
    -Wold-style-cast \
")
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: cmake

```
my-library/cmake/
├── platform/
│   ├── types/Platform
│   │   ├── CMakeLists.txt
│   │   ├── PlatformTypes.fpp
│   │   └── PlatformTypes.h
│   │
│   ├── VxWorks.cmake
│   └── VxWorks-Posix.cmake
│
└── toolchain/
    └── <toolchain>.cmake
```

To generate and build with a particular toolchain, run the following

```
fprime-util generate <toolchain>
fprime-util build <toolchain>
```

NOTE: default toolchain can be set in settings.ini

```
default_toolchain: <toolchain>
```

Then can generate and build like normal

```
fprime-util generate
fprime-util build
```


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Platform Development: MyLibrary

```
my-library/MyLibrary/
├── Components
│   ├── Drv
│   │   └── …
│   ├── Svc
│   │   └── …
│   └── OS
│       └── ...
├── Subtopology
│   └── ...
├── Types
│   └── ...
└── Ports
    └── …
```

- For drivers:
  - Make use of already defined F Prime interfaces
    - https://github.com/nasa/fprime/tree/devel/Drv/Interfaces
  - Try and avoid developing bespoke drivers that directly interface with the hardware memory. Make use of existing API and wrap those in F Prime ports to avoid "reinventing the wheel"
    - May not be possible for baremetal platforms
- Avoid using direct OS-specific function calls unless implementing OSAL features
  - Use the OSAL to make OS calls
- If specific OS implementation is needed, then place OS implementation in the OS directory

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Contribute back

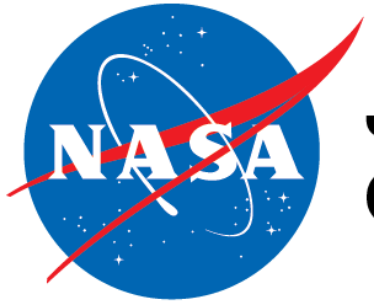
- Create a reference project for your platform that uses your platform library. Use `fprime-zephyr-reference` as examples for the file structure.
- Include build, run, and hardware setup instructions in a README.md.
- Set up CI for your reference project and have CI run the F Prime provided integration test scripts.
- Submit a request to transfer your reference project to the fprime-community by opening a discussion. Please include a description of your platform and links to your reference project repositories. To transfer a repository to fprime-community, you need to transfer ownership of the repository to an fprime maintainer. If accepted, then they will transfer the repository to the fprime-community organization.
- Will help expand the list of supported platforms, and allow for faster deployments for new projects.

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Summary

- F Prime runs on various platforms:
  - X86_64, ARM, RISC-V, Sparc, PowerPc, Snapdragon
- Various references can be found here https://fprime.jpl.nasa.gov/devel/docs/user-manual/framework/supported-platforms/
  - Plans to expand list of references to include HPSC, Hub Pattern, ROS, and cFS
- Use reference projects to help develop your libraries
  - https://github.com/fprime-community/fprime-zephyr-reference
  - https://github.com/fprime-community/fprime-vxworks
- When developing drivers, make use of F Prime interfaces, and try to use existing driver APIs
- Use the OSAL when calling OS functions
- Contribute your platform back to the fprime-community
  - Quick deployments to new projects
  - Faster development and testing
  - Higher reliability through CI-validated platforms
  - Lower cost and reduced duplication of effort
  - Encapsulation of hardware and operating system complexity
  - A personal thanks from myself

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.



Jet Propulsion Laboratory
California Institute of Technology